

SistRUGBY-SLTC

**Sistema de Gestión de Partidos y Estadísticas de Rugby — Santiago Lawn
Tennis Club • Trabajo Práctico N.º 4 (AP4) — Versión final integradora**

Fonzo, Ignacio

Licenciatura en Informática — Universidad Siglo 21 (modalidad a distancia)

Seminario de Práctica — Módulo 4

Junio de 2026 • Repositorio: <https://github.com/nachof9/SistRUGBY-SLTC>

1. Introducción

El presente documento corresponde al Trabajo Práctico N.º 4 (AP4) de la materia Seminario de Práctica de la Licenciatura en Informática de la Universidad Siglo 21. Constituye la **entrega final integradora** del proyecto SistRUGBY-SLTC y, en el marco del Proceso Unificado de Desarrollo (PUD), avanza con la **fase de Transición**: el prototipo deja de operar exclusivamente con datos en memoria y queda **funcional con conexión real a una base de datos MySQL**, tal como se solicitó en la retroalimentación de la entrega anterior.

Por su carácter iterativo e incremental, este trabajo retoma el análisis de negocio (AP1), el diseño y la base de datos (AP2) y el prototipo orientado a objetos (AP3), incorpora las correcciones señaladas por la cátedra y agrega las contribuciones específicas de AP4:

- Persistencia y consulta de datos en MySQL a través de JDBC: establecimiento de conexiones, consultas, actualización de registros y presentación de resultados en la interfaz.
- Selección y justificación del **patrón de diseño** coherente con la arquitectura.
- Correcto manejo de **excepciones** en la interacción con MySQL.
- Inclusión de **clases abstractas** (Persona, EventoPartido).
- Utilización **complementaria de arreglos y de la clase ArrayList**.
- Empleo de **archivos** para guardar y recuperar información (generación de reportes).

2. Resumen del proyecto

2.1 Problemática y justificación (modelo de negocio)

La sección de rugby del SLTC gestiona hoy a sus jugadores, partidos y estadísticas mediante planillas de cálculo individuales y registros en papel, sin integración. Esto provoca pérdida e inconsistencia de datos históricos, imposibilidad de generar reportes ágiles, dificultad para seguir el rendimiento individual y tiempo administrativo excesivo.

Desde la perspectiva del modelo de negocio, el cuerpo técnico y la secretaría deportiva son los **clientes internos**; su **propuesta de valor** es disponer de información oportuna y confiable para la toma de decisiones deportivas (convocatorias, planificación, seguimiento). Las **actividades clave** son el registro del padrón, la carga de partidos y eventos, y la generación de estadísticas; los **recursos clave**, una base de datos centralizada y una aplicación de escritorio sencilla. El proyecto informático se justifica porque elimina los costos ocultos del trabajo manual (retrabajo, errores, pérdida de datos) y habilita un activo reutilizable: el histórico deportivo del club.

2.2 Alcance

El sistema cubre las divisiones juveniles (M15 a M19) y el plantel superior (Pre-Intermedia, Intermedia y Primera). Quedan fuera del alcance las categorías infantiles, la gestión financiera, la transmisión en vivo y la integración con redes sociales. El producto es una aplicación de escritorio con persistencia en MySQL.

2.3 Síntesis de las entregas previas

- **AP1 — Inicio (PUD):** análisis del negocio, requerimientos funcionales (RF01–RF10) y no funcionales (RNF01–RNF07), procesos *as-is*, doce casos de uso y análisis de riesgos.
- **AP2 — Elaboración (PUD):** modelo de dominio, casos de uso detallados, diagramas de secuencia (CU01, CU05, CU07), arquitectura en tres capas, diagramas de clases / componentes / despliegue, modelo relacional normalizado a 3FN y DER.
- **AP3 — Construcción (PUD):** prototipo Java SE aplicando los cuatro pilares de POO, manejo de excepciones, estructuras de datos (lista enlazada, pila, cola, *hashmap*), algoritmos de ordenación (QuickSort) y búsqueda (binaria), y trazabilidad de CU01/CU02/CU07 hasta las pruebas.

Los diagramas UML y los documentos AP1–AP3 se conservan en la carpeta docs/ del repositorio.

3. Aplicación del Proceso Unificado de Desarrollo

El proyecto se desarrolló siguiendo las cuatro fases del PUD, garantizando calidad, escalabilidad y eficiencia mediante iteraciones con artefactos verificables:

Fase

Inicio

Elaboración

Construcción

Transición

La fase de **Transición** que materializa AP4 se concentra en convertir el prototipo en un producto desplegable: se completó la capa de persistencia contra MySQL, se incorporó la generación de reportes en archivos y se afinó el arranque del sistema para tolerar entornos con y sin base de datos.

4. Arquitectura y selección del patrón de diseño

4.1 Arquitectura en tres capas

SistRUGBY-SLTC adopta una **arquitectura en capas** (*Layered Architecture*):

- **Presentación** (presentacion): MenuConsola recolecta entradas y muestra resultados.
- **Negocio** (negocio): clases **Service* con la lógica de aplicación y las validaciones.
- **Persistencia** (persistencia): clases DAO que acceden a MySQL vía JDBC, con ConexionBD (Singleton).

Cada capa conoce únicamente a la inmediatamente inferior, lo que produce bajo acoplamiento y alta cohesión.

4.2 Patrón de diseño seleccionado: DAO (Data Access Object)

La consigna solicita seleccionar y justificar el patrón de diseño coherente con la arquitectura. Se selecciona **DAO (Data Access Object)** como patrón protagonista.

Justificación. El DAO encapsula todo el acceso a datos detrás de una interfaz uniforme (insertar, findById, findAll, actualizar...), de modo que la capa de negocio nunca conoce detalles de JDBC ni SQL. Esto resulta decisivo en este proyecto porque habilita el **modo dual**: la misma firma de método resuelve contra MySQL (rama JDBC) o contra un repositorio en memoria (rama demo) sin que los Service cambien una sola línea. Es, por lo tanto, el patrón que mejor estructura la solución y el que sostiene la característica diferencial del sistema.

Criterio	Aporte del patrón DAO
Pros	Aísla la persistencia; permite intercambiar el origen de datos (MySQL ↔ memoria); facilita el testeo; centraliza el manejo de SQLException.
Contras	Agrega clases por entidad; puede generar código repetitivo de <i>mapeo</i> fila→objeto.
Riesgos	Si las firmas no se respetan, la abstracción se filtra; se mitiga con mapear() privado por DAO.

Patrones de soporte. El diseño se completa con tres patrones auxiliares documentados por Gamma et al. (1994):

- **Singleton** (ConexionBD): una única instancia de conexión compartida por todos los DAOs; getInstance() es synchronized (seguridad ante hilos).
- **Factory Method** (EventoPartido.crear): decide qué subclase concreta de evento instanciar a partir del enum Tipo, evitando que el cliente acople con las siete clases.
- **Layered Architecture**: patrón arquitectónico macro que las tres capas materializan.

5. Base de datos MySQL

5.1 Modelo relacional

El esquema se normalizó hasta la **Tercera Forma Normal (3FN)**. Se compone de ocho tablas alineadas exactamente con las entidades del modelo Java: usuarios, categorias, clubes, temporadas, jugadores, partidos, partido_plantel y eventos_partido. Las relaciones se implementan con claves foráneas e integridad referencial sobre el motor InnoDB (ON UPDATE CASCADE, ON DELETE RESTRICT/CASCADE según corresponda). El DER se conserva en docs/diagramas/der.png.

Corrección incorporada en AP4. En entregas previas el script schema.sql había quedado desalineado respecto del código (usaba divisiones/id_division y hashes bcrypt de ejemplo, mientras el código usaba categorias/id_categoria y PBKDF2). En AP4 se unificó el esquema con la capa de persistencia, condición necesaria para que el modo MySQL funcione.

5.2 Fragmento del DDL

```
CREATE TABLE jugadores (  
  id_jugador INT AUTO_INCREMENT PRIMARY KEY,  
  nombre VARCHAR(80) NOT NULL,  
  apellido VARCHAR(80) NOT NULL,  
  dni VARCHAR(10) NOT NULL UNIQUE,  
  fecha_nacimiento DATE NOT NULL,  
  posicion VARCHAR(30),  
  id_categoria INT NOT NULL,  
  estado ENUM('activo','inactivo') NOT NULL DEFAULT 'activo',  
  fecha_alta DATE NOT NULL DEFAULT (CURDATE()),  
  fecha_baja DATE,  
  CONSTRAINT fk_jug_cat FOREIGN KEY (id_categoria)  
    REFERENCES categorias (id_categoria)  
    ON UPDATE CASCADE ON DELETE RESTRICT  
) ENGINE = InnoDB;  
  
CREATE TABLE eventos_partido (  
  id_evento INT AUTO_INCREMENT PRIMARY KEY,  
  id_partido INT NOT NULL,  
  id_jugador INT NOT NULL,  
  tipo_evento ENUM('TRY','CONVERSION','PENAL','DROP',  
    'TARJETA_AMARILLA','TARJETA_ROJA','SUSTITUCION') NOT  
  NULL,  
  minuto INT,  
  descripcion VARCHAR(200),  
  CONSTRAINT fk_ev_partido FOREIGN KEY (id_partido)  
    REFERENCES partidos (id_partido) ON UPDATE CASCADE ON DELETE  
  CASCADE,  
  CONSTRAINT fk_ev_jugador FOREIGN KEY (id_jugador)  
    REFERENCES jugadores (id_jugador) ON UPDATE CASCADE ON  
  DELETE RESTRICT  
) ENGINE = InnoDB;
```

El script completo (sql/schema.sql) crea las ocho tablas con sus claves e índices; sql/datos_iniciales.sql carga los datos de prueba (ocho categorías, cinco clubes, tres temporadas, tres usuarios, cinco jugadores y un partido de muestra) e incluye consultas SELECT representativas de estadísticas y reportes.

6. Persistencia y consulta con JDBC

6.1 Conexión: Singleton y modo dual robusto

ConexionBD aplica Singleton y, en AP4, incorpora un **modo dual robusto**: si la variable de entorno DB_PASS está definida, intenta abrir la conexión JDBC real; si no lo está, o si la conexión falla (servidor apagado, credenciales incorrectas, driver ausente), el sistema **cae automáticamente a un repositorio en memoria** informando el motivo, en lugar de abortar. Así, la corrección puede realizarse con o sin MySQL instalado.

```
private ConexionBD() {  
  String url = System.getenv().getOrDefault("DB_URL", URL_DEFAULT);
```

```

String user = System.getenv().getOrDefault("DB_USER", USER_DEFAULT);
String pass = System.getenv().getOrDefault("DB_PASS", "");
if (pass.isEmpty()) { // sin credenciales -> demo
    this.modosMemoria = true;
    this.motivoFallback = "DB_PASS no definida (modo demo en memoria).";
    return;
}
try {
    this.connection = DriverManager.getConnection(url, user, pass);
} catch (SQLException e) { // fallback robusto, no aborta
    this.modosMemoria = true;
    this.motivoFallback = "No se pudo conectar a MySQL (" + e.getMessage()
        + "). Se activa modo memoria.";
}
}

```

6.2 Operaciones JDBC en los DAOs

Cada DAO resuelve la operación contra MySQL con PreparedStatement (consultas parametrizadas que previenen inyección SQL) y, en modo demo, contra RepositorioMemoria. Se ejemplifican las cuatro operaciones requeridas: establecer conexión, consultar, actualizar y presentar resultados.

Alta (INSERT) con recuperación de clave generada — JugadorDAO:

```

String sql = "INSERT INTO jugadores (nombre, apellido, dni, fecha_nacimiento,
"
    + "posicion, id_categoria, estado, fecha_alta) "
    + "VALUES (?, ?, ?, ?, ?, ?, 'activo', CURDATE())";
try (PreparedStatement ps = bd.getConnection().prepareStatement(
    sql, Statement.RETURN_GENERATED_KEYS)) {
    ps.setString(1, j.getNombre());
    ps.setString(2, j.getApellido());
    ps.setString(3, j.getDni());
    ps.setDate(4, Date.valueOf(j.getFechaNacimiento()));
    ps.setString(5, j.getPosicion());
    ps.setInt(6, j.getIdCategoria());
    ps.executeUpdate();
    try (ResultSet rs = ps.getGeneratedKeys()) {
        if (rs.next()) j.setId(rs.getInt(1));
    }
}

```

Consulta (SELECT) y mapeo fila→objeto — JugadorDAO:

```

String sql = "SELECT * FROM jugadores WHERE dni = ?";
try (PreparedStatement ps = bd.getConnection().prepareStatement(sql)) {
    ps.setString(1, dni);
    try (ResultSet rs = ps.executeQuery()) {
        if (rs.next()) return mapear(rs); // ResultSet -> Jugador
    }
}

```

Actualización (UPDATE) — baja lógica que preserva el historial:

```
String sql = "UPDATE jugadores SET estado='inactivo', fecha_baja=CURDATE()
"
    + "WHERE id_jugador = ?";
try (PreparedStatement ps = bd.getConnection().prepareStatement(sql)) {
    ps.setInt(1, idJugador);
    ps.executeUpdate();
}
```

Transacción (atomicidad) — carga del plantel en PartidoDAO:

```
con.setAutoCommit(false);
try (PreparedStatement ps = con.prepareStatement(sql)) {
    for (PlantelPartido pp : plantel) { ps.setInt(1, idPartido); /* ... */
ps.addBatch(); }
    ps.executeBatch();
    con.commit();
} catch (SQLException e) {
    con.rollback(); // se deshace la carga parcial ante cualquier fallo
    throw e;
} finally { con.setAutoCommit(true); }
```

6.3 Polimorfismo en la persistencia de eventos

EventoDAO guarda el discriminador `tipo_evento = evento.getTipo().name()` y, al leer, reconstruye la subclase concreta mediante la fábrica polimórfica `EventoPartido.crear(...)`. Ni el DAO ni la base de datos conocen las siete clases concretas:

```
private EventoPartido mapear(ResultSet rs) throws SQLException {
    EventoPartido.Tipo tipo =
EventoPartido.Tipo.valueOf(rs.getString("tipo_evento"));
    EventoPartido e = EventoPartido.crear(tipo, rs.getInt("id_partido"),
        rs.getInt("id_jugador"), rs.getInt("minuto"));
    e.setId(rs.getInt("id_evento"));
    e.setDescripcion(rs.getString("descripcion"));
    return e;
}
```

7. Manejo de excepciones

El sistema distingue **excepciones de dominio** (reglas de negocio violadas) de **excepciones técnicas** (errores de E/S o de la base de datos):

Excepción

DniDuplicadoException

JugadorNoEncontradoException

CredencialesInvalidasException

DatosInvalidosException

SQLException

IOException

En la interacción con MySQL, las operaciones declaran `SQLException`; la capa de presentación captura por especificidad (de la más concreta a la más genérica) y, ante el fallo de conexión, el modo dual evita la caída del sistema. La escritura de reportes maneja `IOException` de forma diferenciada.

8. Clases abstractas y pilares de POO

El prototipo materializa los cuatro pilares de la programación orientada a objetos mediante dos jerarquías con **clases abstractas**:

- **Persona** (abstracta) → Jugador, Usuario (herencia). Declara el contrato `descripcionCorta()`.
- **EventoPartido** (abstracta) → Try, Conversion, Penal, Drop, TarjetaAmarilla, TarjetaRoja, Sustitucion. Declara `calcularPuntos()` y `getTipo()`.

Abstracción y **herencia** se observan en esas jerarquías; el **encapsulamiento**, en los atributos `private/protected` con `getters/setters` y enums tipados; el **polimorfismo**, en `calcularPuntos()`, que devuelve el valor correcto según el tipo concreto sin recurrir a `instanceof`:

```
public int recalcularPuntosSLTC() {
    int total = 0;
    for (EventoPartido e : eventos) total += e.calcularPuntos(); // cada subclase responde
    return total;
}
```

9. Utilización complementaria de arreglos y ArrayList

La consigna exige emplear **arreglos** y la clase **ArrayList** de forma complementaria. El sistema usa cada estructura donde es óptima:

- **ArrayList (tamaño dinámico)** para el ranking de jugadores, cuya cantidad de elementos (jugadores con eventos) se desconoce de antemano y crece según los datos.
- **Arreglo int[] (tamaño fijo)** para el resumen de eventos por tipo: su tamaño es conocido en tiempo de compilación (la cantidad de valores del enum `Tipo`) y el acceso es $O(1)$ indexando por `ordinal()`, sin *boxing*.

```
public int[] conteoGlobalPorTipo() throws SQLException {
    int[] conteo = new int[EventoPartido.Tipo.values().length]; // ARREGLO fijo
    for (Partido p : partidoDao.findAll()) {
        for (EventoPartido e : eventoDao.findByPartido(p.getId())) {
            conteo[e.getTipo().ordinal()]++; // acceso O(1)
        }
    }
    return conteo;
}
// El ranking, en cambio, se devuelve como ArrayList<FilaRanking> (tamaño dinámico).
```


10. Empleo de archivos

ReporteService (CU09) exporta el ranking y el resumen de eventos a un archivo de texto. La escritura usa try-with-resources sobre un BufferedWriter (cierre automático del recurso) y maneja IOException, combinando el ArrayList del ranking con el arreglo int[] del resumen:

```
try (BufferedWriter w = Files.newBufferedWriter(destino,
StandardCharsets.UTF_8)) {
    w.write("RANKING DE JUGADORES POR PUNTOS"); w.newLine();
    for (FilaRanking f : ranking) { /* ... */ w.newLine(); }
    EventoPartido.Tipo[] tipos = EventoPartido.Tipo.values(); // arreglo
    for (int i = 0; i < tipos.length; i++)
        w.write(String.format("%-18s : %d", tipos[i], conteoPorTipo[i]));
}
```

11. Estructuras de datos y algoritmos

Se conservan, del prototipo AP3, las estructuras y algoritmos con justificación funcional:

Estructura / algoritmo

LinkedList<EventoPartido>

ArrayDeque (pila LIFO)

Queue (FIFO)

HashMap

QuickSort (mediana de tres)

Búsqueda binaria

12. Plan de pruebas y evidencia

12.1 Estrategia

Se combinaron pruebas funcionales de caja negra, pruebas de integración (comunicación entre capas), pruebas de seguridad (verificación de hash) y pruebas de regresión. El prototipo se ejecuta de forma scripteada alimentando el menú con un archivo de entrada (smoke_input.txt).

12.2 Resultados

ID

PT01

PT02

PT03

PT04

PT05

PT06

PT07
PT08
PT09
PT10
PT11
PT12
PT13

12.3 Evidencia de ejecución (modo demo)

[INFO] DB_PASS no definida (modo demo en memoria).

[INFO] Modo demo (en memoria). Cargando datos de prueba...

--- CU07 Registrar evento (polimórfico) ---

OK. Evento registrado: TRY en minuto 12 (jugador #3) [5 pts]

OK. Evento registrado: CONVERSION en minuto 13 (jugador #3) [2 pts]

OK. Evento registrado: TRY en minuto 28 (jugador #1) [5 pts]

OK. Evento registrado: PENAL en minuto 55 (jugador #3) [3 pts]

--- Ranking por puntos (POLIMORFISMO en calcularPuntos) ---

JUGADOR	TRY	CONV	PEN	DROP	PUNTOS
---------	-----	------	-----	------	--------

Casas, Federico	1	1	1	0	10
-----------------	---	---	---	---	----

Pereyra, Rodrigo	1	0	0	0	5
------------------	---	---	---	---	---

12.4 Evidencia del modo MySQL

El proyecto se compiló sin errores con JDK 17 (52 clases). El controlador mysql-connector-j 8.3.0 se carga correctamente y la conexión se intenta contra jdbc:mysql://localhost:3306/sistrugby_sltc; cuando el servidor no está disponible, el sistema informa el motivo y conmuta a memoria (prueba PT13). Los hashes PBKDF2 incluidos en datos_iniciales.sql fueron **generados y verificados** (cada uno valida su contraseña y rechaza una incorrecta), de modo que el inicio de sesión opera también en modo MySQL. Las sentencias SQL de cada DAO fueron contrastadas, columna por columna, contra schema.sql.

13. Atención de las observaciones previas

- **AP3 (docente):** “en la cuarta entrega debe ya estar con conexión a la base... algo funcional y con conexión a la base de datos.” → AP4 completa el JDBC de **todos** los DAOs (eventos, partidos, plantel, categorías, clubes y temporadas), unifica schema.sql con el código y entrega hashes PBKDF2 reales para el login en MySQL.
- **POO:** se ratifican encapsulamiento, herencia, polimorfismo y abstracción, con revisión de constructores (por defecto, de alta y completo) y de la creación de objetos.
- **AP2:** la portada no contiene encabezado ni pie de página y se mantiene la trazabilidad CU → código → pruebas.

14. Repositorio y guía de ejecución

Repositorio público: <https://github.com/nachof9/SistRUGBY-SLTC>

Modo demo (sin MySQL):

```
javac -d out -encoding UTF-8 $(find src/main/java -name "*.java")
java -cp out com.sltc.sistrugby.Main
```

Modo producción (con MySQL):

```
mysql -u root -p < sql/schema.sql
mysql -u root -p < sql/datos_iniciales.sql
export DB_PASS='tu_password'
java -cp out:lib/mysql-connector-j-8.3.0.jar com.sltc.sistrugby.Main
```

Credenciales de prueba: admin/admin2026, entrenador1/rugby2026, secretario1/sltc2026.

15. Conclusiones

AP4 cierra el ciclo del PUD para SistRUGBY-SLTC entregando un producto **funcional y con conexión real a MySQL**. Se completó la capa de persistencia en todos los DAOs, se seleccionó y justificó el patrón **DAO** como eje de la solución (con Singleton y Factory Method de apoyo), se incorporaron arreglos y ArrayList de manera complementaria, y se agregó la generación de reportes en archivos. El modo dual robusto permite ejecutar el sistema con o sin base de datos, facilitando tanto la operación en el club como la corrección académica. La arquitectura en capas y la separación que impone el patrón DAO dejan el sistema preparado para futuras extensiones (interfaz gráfica Swing, exportación a PDF/CSV, pruebas unitarias con JUnit) sin reescribir la lógica existente.

16. Referencias

Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley.

Jacobson, I., Booch, G., & Rumbaugh, J. (1999). *The Unified Software Development Process*. Addison-Wesley.

Kendall, K., & Kendall, J. (2011). *Análisis y diseño de sistemas* (8.ª ed.). Pearson Educación.

Larman, C. (2003). *UML y patrones* (2.ª ed.). Pearson Prentice Hall.

National Institute of Standards and Technology. (2010). *NIST Special Publication 800-132: Recommendation for Password-Based Key Derivation*. NIST.

Oracle Corporation. (2024). *Java SE 17 Documentation*.
<https://docs.oracle.com/en/java/javase/17/>

MySQL AB. (2024). *MySQL 8.0 Reference Manual*.
<https://dev.mysql.com/doc/refman/8.0/en/>